8

Classes e objetos: Um exame mais profundo

© 2005 by Pearson Education do Brasil

OBJETIVOS

- Neste capítulo, você aprenderá:
- Encapsulamento e ocultamento de dados.
- As noções de abstração de dados e tipos de dados abstratos (*abstract data types* – ADTs)
- Como utilizar a palavra-chave `this`.
- Como utilizar variáveis e métodos `static`.
- Como importar membros `static` de uma classe.
- Como utilizar o tipo `enum` para criar conjuntos de constantes com identificadores únicos.
- Como declarar constantes `enum` com parâmetros.

© 2005 by Pearson Education do Brasil

Sumário

8.1 Introdução

8.2 Estudo de caso da classe `Time`

8.3 Escopo de classe

8.4 Referenciando membros do objeto atual com a referência `this`

8.5 Estudo de caso da classe `Time`: Construtores sobrecarregados

8.6 Construtores-padrão e sem argumentos

8.7 Notas sobre os métodos *Set* e *Get*

8.8 Composição

8.9 Enumerações

8.10 Coleta de lixo e o método `finalize`

© 2005 by Pearson Education do Brasil

Sumário

8.11 Membros da classe `static`

8.12 Import `static`

8.13 Variáveis de instância `final`

8.14 Capacidade de reutilização de software

8.15 Abstração de dados e encapsulamento

8.16 Estudo de caso da classe `Time`: Controlando o acesso a membros

8.17 Acesso de pacote

8.18 (Opcional) Estudo de caso de GUI e imagens gráficas: Utilizando objetos com imagens gráficas

8.19 (Opcional) Estudo de caso de engenharia de software: Começando a programar as classes do sistema ATM

© 2005 by Pearson Education do Brasil

8.2 Estudo de caso da classe `Time`

- **Serviços `public` (ou interface `public`)**
  - **Métodos `public` disponíveis para uso por um cliente.**
- **Se uma classe não definir um construtor, o compilador fornecerá um construtor-padrão.**
- **Variáveis de instância:**
  - **Podem ser inicializadas quando são declaradas ou em um construtor.**
  - **Devem manter valores consistentes (válidos).**

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.1

Os métodos que modificam os valores de variáveis `private` devem verificar se os novos valores pretendidos são adequados. Se não forem, os métodos *set* devem colocar as variáveis `private` em um estado consistente apropriado.

© 2005 by Pearson Education do Brasil

Resumo

Time1.java

(1 de 2)

```
// Fig. 8.1: Time1.java
// Declaração de classe Time1 mantém a data/hora no formato de 24 horas.

public class Time1
{
   private int hour;   // 0 – 23
   private int minute; // 0 – 59
   private int second; // 0 – 59

   // configura um novo valor de data/hora usando UTC; assegura que
   // os dados permaneçam consistentes configurando valores inválidos como zero
   public void setTime( int h, int m, int s )

      hour = ( ( h >= 0 && h < 24 ) ? h : 0 );   // valida horas
      minute = ( ( m >= 0 && m < 60 ) ? m : 0 ); // validate minutos
      second = ( ( s >= 0 && s < 60 ) ? s : 0 ); // validate segundos
   } // fim do método setTime

```

Variáveis de instância **private**

Declara o método **public setTime**

Valida valores de parâmetro antes de configurar variáveis de instância

© 2005 by Pearson Education do Brasil

Resumo

Time1.java

(2 de 2)

```
19    // converte em String no formato de data/hora universal (HH:MM:SS)
20    public String toUniversalString()
21    {
22       return String.format( "%02d:%02d:%02d", hour, minute, second );
23    } // fim do método toUniversalString
24
25    // converte em String no formato padrão de data/hora (H:MM:SS AM ou PM)
26    public String toString()
27    {
28       return String.format( "%d:%02d:%02d %s",
29          ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 ),
30          minute, second, ( hour < 12 ? "AM" : "PM" ) );
31    } // fim do método toString
32 } // fim da classe Time1
```

Strings **format**

© 2005 by Pearson Education do Brasil

8.2 Estudo de caso da classe `Time` (*Continuação*)

- **Método `String.format`:**
  - **Semelhante a `printf`, exceto que retorna uma string formatada em vez de exibi-la em uma janela de comando.**
- **`new` invoca implicitamente o construtor-padrão de `Time1`, uma vez que `Time1` não declara nenhum construtor.**

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.2

As classes simplificam a programação, porque o cliente pode utilizar somente os métodos `public` expostos pela classe. Normalmente, esses métodos são direcionados aos clientes em vez de direcionados à implementação. Os clientes não estão cientes de, nem envolvidos em, uma implementação da classe. Os clientes geralmente se preocupam com *o que* a classe faz, mas não *como* a classe faz isso.

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.3

As interfaces mudam com menos freqüência que as implementações. Quando uma implementação muda, o código dependente de implementação deve alterar correspondentemente. Ocultar a implementação reduz a possibilidade de que outras partes do programa irão se tornar dependentes dos detalhes sobre a implementação da classe.

© 2005 by Pearson Education do Brasil

Resumo

Time1Test.java

(1 de 2)

```
// Fig. 8.2: Time1Test.java
// Objeto Time1 usado em um aplicativo.

public class Time1Test
{
   public static void main( String args[] )
   {
      // cria e inicializa um objeto Time1
      Time1 time = new Time1(); // invoca o construtor Time1

      // gera representações de string da data/hora
      System.out.print( "The initial universal time is: " );
      System.out.println( time.toUniversalString() );
      System.out.print( "The initial standard time is: " );
      System.out.println( time.toString() );
      System.out.println(); // gera saída de uma linha em branc

```

Cria um objeto **Time1**

Chama o método **toUniversalString**

Chama o método **toString**

© 2005 by Pearson Education do Brasil

Resumo

Time1Test.java

(2 de 2)

```
      // altera a data/hora e gera saída da data/h
      time.setTime( 13, 27, 6 );
      System.out.print( "Universal time after setTime is: " );
      System.out.println( time.toUniversalString() );
      System.out.print( "Standard time after setTime is: " );
      System.out.println( time.toString() );
      System.out.println(); // gera saída de uma linha em branco

      // configura data/hora com valores inválidos; gera saída da data/hora
      time.setTime( 99, 99, 99 );
      System.out.println( "After attempting invalid settings:" );
      System.out.print( "Universal time: " );
      System.out.println( time.toUniversalString() );
      System.out.print( "Standard time: " );
      System.out.println( time.toString() );
   } // fim de main
} // fim da classe Time1Test
```

Chama o método **setTime**

Chama o método **setTime** com valores inválidos

```
The initial universal time is: 00:00:00
The initial standard time is: 12:00:00 AM

Universal time after setTime is: 13:27:06
Standard time after setTime is: 1:27:06 PM

After attempting invalid settings:
Universal time: 00:00:00
Standard time: 12:00:00 AM
```

© 2005 by Pearson Education do Brasil

8.3 Escopo de classe

- **A interface pública de uma classe:**
  - **Métodos `public` apresentam uma visualização dos serviços que a classe fornece aos clientes da classe.**
- **Detalhes da implementação de uma classe:**
  - **Variáveis `private` e métodos `private` não são acessíveis aos clientes da classe.**

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.1

Uma tentativa por um método que não é membro de uma classe de acessar um membro `private` dessa classe é um erro de compilação.

© 2005 by Pearson Education do Brasil

Resumo

```
1  // Fig. 8.3: MemberAccessTest.java
2  // Membros privados da classe Time1 não são acessíveis.
3  public class MemberAccessTest
4  {
5     public static void main( String args[] )
6     {
7        Time1 time = new Time1(); // cria e inicializa o objeto Time1
8
9        time.hour = 7;     // error: hour has private access in Time1
10       time.minute = 15; // error: minute has private access in Time1
11       time.second = 30; // error: second has private access in Time1
12    } // fim de main
13 } // fim da classe MemberAccessTest
```

Tentando acessar variáveis de instância **`private`**

```
MemberAccessTest.java:9: hour has private access in Time1
      time.hour = 7;    // error: hour has private access in Time1
          ^
MemberAccessTest.java:10: minute has private access in Time1
      time.minute = 15; // error: minute has private access in Time1
          ^
MemberAccessTest.java:11: second has private access in Time1
      time.second = 30; // error: second has private access in Time1
          ^
3 errors
```

`MemberAccessTest`

`.java`

© 2005 by Pearson Education do Brasil

8.4 Referenciando membros do objeto atual com a referência `this`

- **A referência `this`:**
  - **Qualquer objeto pode acessar uma referência dele mesmo com a palavra-chave `this`.**
  - **Métodos não-estáticos utilizam implicitamente `this` ao referenciar variáveis de instância do objeto e outros métodos.**
  - **Pode ser utilizada para acessar variáveis de instância quando elas estão sombreadas por variáveis locais ou parâmetros de método.**
- **Um arquivo `.java` pode conter mais de uma classe:**
  - **Mas somente uma classe em cada arquivo `.java` pode ser `public`.**

© 2005 by Pearson Education do Brasil

Resumo

ThisTest.java

(1 de 2)

```
// Fig. 8.4: ThisTest.java
// this utilizado implícita e explicitamente p/ referência a membros de um objeto.

public class ThisTest
{
   public static void main( String args[] )
   {
      SimpleTime time = new SimpleTime( 15, 30, 19 );
      System.out.println( time.buildString() );
   } // fi de main
} // fim da classe ThisTest

// class SimpleTime demonstrata a referência "this"
class SimpleTime
{
   private int hour;   // 0-23
   private int minute; // 0-59
   private int second; // 0-59

   // se o construtor usa nomes de parâmetro idênticos aos
   // nomes de variáveis de instância a referência "this" será
   // necessária para distinguir entre nomes
   public SimpleTime( int hour, int minute, int second )
   {
      this.hour = hour;       // configura a hora do objeto "this"
      this.minute = minute;   // configura o minuto do objeto "this"
      this.second = second;   // configura o segundo do objeto "this"
   } // fim do construtor SimpleTime

```

Cria um novo objeto **`SimpleTime`**

Declara variáveis de instância

Variáveis de instância sombreiam parâmetros de método

Utilizando `this` para acessar as variáveis de instância do objeto

© 2005 by Pearson Education do Brasil

Resumo

ThisTest.java

(2 de 2)

```
   // utiliza “this” explícito e implícito para chamar toUniversalString
   public String buildString()
   {
      return String.format( "%24s: %s\n%24s: %s",
         "this.toUniversalString()", this.toUniversalString(),
         "toUniversalString()", toUniversalString() );
   } // fim do método buildString

   // converte em String no formato de data/hora universal (HH
   public String toUniversalString()
   {
      // “this” não é requerido aqui para acessar variáveis de instância,
      // porque o método não tem variáveis locais com os mesmos
      // nomes das variáveis de instância
      return String.format( "%02d:%02d:%02d",
         this.hour, this.minute, this.second );
   } // fim do método toUniversalString
} // fim da classe SimpleTime
```

Utilizando **this** explícita e implicitamente para chamar **toUniversalString**

A utilização de **this** não é necessária aqui

```
this.toUniversalString(): 15:30:19
     toUniversalString(): 15:30:19
```

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.2

Freqüentemente é um erro de lógica quando um método contém um parâmetro ou variável local com o mesmo nome de um campo da classe.

Nesse caso, utilize a referência `this` se desejar acessar o campo da classe — caso contrário, o parâmetro ou variável local do método será referenciado.

© 2005 by Pearson Education do Brasil

Dica de prevenção de erro 8.1

Evite nomes de parâmetros ou variáveis locais nos métodos que conflitem com nomes dos campos. Isso ajuda a evitar bugs sutis, difíceis de corrigir.

© 2005 by Pearson Education do Brasil

Dica de desempenho 8.1

O Java economiza memória mantendo somente uma cópia de cada método por classe — esse método é invocado por todos os objetos dessa classe. Cada objeto, por outro lado, tem sua própria cópia das variáveis de instância da classe (isto é, campos não-`static`). Cada método da classe utiliza implicitamente `this` para determinar o objeto específico da classe a manipular.

© 2005 by Pearson Education do Brasil

8.5 Estudo de caso da classe `Time`: Construtores sobrecarregados

- **Construtores sobrecarregados:**
  - **Forneça múltiplas definições de construtor com diferentes assinaturas.**
- **Construtor sem argumento:**
  - **Um construtor invocado sem argumentos.**
- **A referência `this` pode ser utilizada para invocar um outro construtor:**
  - **Permitido somente como a primeira instrução no corpo de um construtor.**

© 2005 by Pearson Education do Brasil

Resumo

Time2.java

(1 de 4)

```
// Fig. 8.5: Time2.java
// Declaração da classe Time2 com construtores sobrecarregados.

public class Time2
{
   private int hour;   // 0 - 23
   private int minute; // 0 - 59
   private int second; // 0 - 59

   // construtor sem argumento Time2 : inicializa cada variável de instância
   // com zero; assegura que objetos Time2 iniciam em um estado consistente
   public Time2()
   {
      this( 0, 0, 0 ); // invoca o construtor Time2 com três argumentos
   } // fim do construtor sem argumento Time2

   // Construtor Time2: hora fornecida, minuto e segundo padronizados para 0
   public Time2( int h )
   {
      this( h, 0, 0 ); // invoca o construtor Time2 com três argumentos
   } // end Time2 one-argument constructor

   // Construtor Time2: hora e minuto fornecidos, segundo padronizado para 0
   public Time2( int h, int m )
   {
      this( h, m, 0 ); // invoca o construtor Time2 com três argumentos
   } // fim do construtor de um argumento Time2

```

Construtor sem argumento

Invoca construtor com três argumentos

© 2005 by Pearson Education do Brasil

Resumo

Time2.java

(2 de 4)

```
   // Construtor Time2: hour, minute e second fornecidos
   public Time2( int h, int m, int s )
   {
      setTime( h, m, s ); // invoca setTime para validar data/hora
   } // fim do construtor Time2 de três argumentos

   // Construtor Time2: outro objeto Time2 fornecido
   public Time2( Time2 time )
   {
      // invoca construtor Time2 de tr
      this( time.getHour(), time.getMinute(), time.getSecond() );
   } // fim do construtor Time2 com um argumento de objeto Time2

   // Métodos set
   // configura um novo valor de data/hora usando UTC; assegura
   // os dados permaneçam consistentes configurando valores inválidos como zero
   public void setTime( int h, int m, int s )
   {
      setHour( h );   // configura hour
      setMinute( m ); // configura minute
      setSecond( s ); // configura second
   } // fim do método setTime

```

Chama o método **setTime**

Construtor recebe uma referência a um outro objeto **Time2** como um parâmetro

Poderia ter acessado diretamente variáveis de instância do objeto **time** aqui

© 2005 by Pearson Education do Brasil

Resumo

Time2.java

(3 de 4)

```
   // valida e configura a hora
   public void setHour( int h )
   {
      hour = ( ( h >= 0 && h < 24 ) ? h : 0 );
   } // fim do método setHour

   // validate and set minute
   public void setMinute( int m )
   {
      minute = ( ( m >= 0 && m < 60 ) ? m : 0 );
   } // fim do método setMinute

   // valida e configura os segundos
   public void setSecond( int s )
   {
      second = ( ( s >= 0 && s < 60 ) ? s : 0 );
   } // fim do método setSecond

   // Métodos get
   // obtém valor da hora
   public int getHour()
   {
      return hour;
   } // fim do método getHour

```

© 2005 by Pearson Education do Brasil

Resumo

Time2.java

(4 de 4)

```
   // obtém valor dos minutos
   public int getMinute()
   {
      return minute;
   } // fim do método getMinute

   // obtém valor dos segundos
   public int getSecond()
   {
      return second;
   } // fim do método getSecond

   // converte em String no formato de data/hora universal (HH:MM:SS)
   public String toUniversalString()
   {
      return String.format(
         "%02d:%02d:%02d", getHour(), getMinute(), getSecond() );
   } // fim do método do toUniversalString

   // converte em String no formato padrão de data/hora (H:MM:SS AM ou PM)
   public String toString()
   {
      return String.format( "%d:%02d:%02d %s",
         ( (getHour() == 0 || getHour() == 12) ? 12 : getHour() % 12 ),
         getMinute(), getSecond(), ( getHour() < 12 ? "AM" : "PM" ) );
   } // fim do método toString
} // fim da classe Time2
```

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.3

É um erro de sintaxe se `this` for utilizado no corpo de um construtor para chamar um outro construtor da mesma classe se essa chamada não for a primeira instrução no construtor. Também é um erro de sintaxe se um método tentar invocar um construtor diretamente via `this`.

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.4

Um construtor pode chamar métodos da classe. Esteja ciente de que as variáveis de instância talvez ainda não estejam em um estado consistente, porque o construtor está no processo de inicialização do objeto. Utilizar variáveis de instância antes de elas serem inicializadas adequadamente é um erro de lógica.

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.4

Quando um objeto de uma classe contém uma referência a um outro objeto da mesma classe, o primeiro objeto pode acessar todos os dados e métodos do segundo objeto (incluindo aqueles que são `private`).

© 2005 by Pearson Education do Brasil

8.5 Estudo de caso da classe `Time`: Construtores sobrecarregados (*Cont.*)

- **Utilizando métodos *set*:**
  - **Fazer com que os construtores utilizem métodos *set* para modificar variáveis de instância, em vez de modificá-las diretamente, simplifica a alteração da implementação.**

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.5

Ao implementar um método de uma classe, utilize os métodos *set* e *get* da classe para acessar os dados `private` da classe.

Isso simplifica a manutenção do código e reduz a probabilidade de erros.

© 2005 by Pearson Education do Brasil

Resumo

Chama construtores sobrecarregados

Time2Test.java

(1 de 3)

```java
// Fig. 8.6: Time2Test.java
// Construtores utilizados para inicializar objetos Time2.

public class Time2Test
{
   public static void main( String args[] )
   {
      Time2 t1 = new Time2();           // 00:00:00
      Time2 t2 = new Time2( 2 );        // 02:00:00
      Time2 t3 = new Time2( 21, 34 );   // 21:34:00
      Time2 t4 = new Time2( 12, 25, 42 ); // 12:25:42
      Time2 t5 = new Time2( 27, 74, 99 ); // 00:00:00
      Time2 t6 = new Time2( t4 );       // 12:25:42

      System.out.println( "Constructed with:" );
      System.out.println( "t1: all arguments defaulted" );
      System.out.printf( "   %s\n", t1.toUniversalString() );
      System.out.printf( "   %s\n", t1.toString() );

```

© 2005 by Pearson Education do Brasil

Resumo

```
      System.out.println(
         "t2: hour specified; minute and second defaulted" );
      System.out.printf( "   %s\n", t2.toUniversalString() );
      System.out.printf( "   %s\n", t2.toString() );

      System.out.println(
         "t3: hour and minute specified; second defaulted" );
      System.out.printf( "   %s\n", t3.toUniversalString() );
      System.out.printf( "   %s\n", t3.toString() );

      System.out.println( "t4: hour, minute and second specified" );
      System.out.printf( "   %s\n", t4.toUniversalString() );
      System.out.printf( "   %s\n", t4.toString() );

      System.out.println( "t5: all invalid values specified" );
      System.out.printf( "   %s\n", t5.toUniversalString() );
      System.out.printf( "   %s\n", t5.toString() );

```

Time2Test.java

(2 de 3)

© 2005 by Pearson Education do Brasil

Resumo

```
38        System.out.println( "t6: Time2 object t4 specified" );
39        System.out.printf( "   %s\n", t6.toUniversalString() );
40        System.out.printf( "   %s\n", t6.toString() );
41     } // fim de main
42 } // fim da classe Time2Test
```

```
t1: all arguments defaulted
   00:00:00
   12:00:00 AM
t2: hour specified; minute and second defaulted
   02:00:00
   2:00:00 AM
t3: hour and minute specified; second defaulted
   21:34:00
   9:34:00 PM
t4: hour, minute and second specified
   12:25:42
   12:25:42 PM
t5: all invalid values specified
   00:00:00
   12:00:00 AM
t6: Time2 object t4 specified
   12:25:42
   12:25:42 PM
```

Time2Test.java

(3 de 3)

© 2005 by Pearson Education do Brasil

8.6 Construtores-padrão e sem argumentos

- **Cada classe deve ter pelo menos um construtor.**
  - **Se nenhum construtor for declarado, o compilador criará um construtor-padrão.**
    - **Não recebe nenhum argumento e inicializa variáveis de instância de acordo com seus valores iniciais especificados nas suas declarações ou de acordo com seus valores-padrão.**
      - **Valores-padrão são zero para tipos numéricos primitivos, `false` para valores `boolean` e `null` para referências.**
  - **Se os construtores forem declarados, a inicialização-padrão para objetos da classe será realizada por um construtor sem argumentos (se um for declarado).**

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.5

Se uma classe tiver construtores, mas nenhum dos construtores `public` for um construtor sem argumentos, e um programa tentar chamar um construtor sem argumentos para inicializar um objeto da classe, ocorrerá um erro de compilação. Um construtor pode ser chamado sem argumentos somente se a classe não tiver nenhum construtor (nesse caso, o construtor-padrão é chamado) ou se a classe tiver um construtor `public` sem argumentos.

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.6

O Java permite que outros métodos da classe além dos seus construtores tenham o mesmo nome da classe e que especifiquem tipos de retorno. Esses métodos não são construtores e não serão chamados quando um objeto da classe for instanciado.

O Java determina quais métodos são construtores localizando os métodos que têm o mesmo nome da classe e que não especificam um tipo de retorno.

© 2005 by Pearson Education do Brasil

8.7 Notas sobre os métodos *Set* e *Get*

- **Métodos *set*:**
  - **Também conhecidos como métodos modificadores.**
  - **Atribuem valores a variáveis de instância.**
  - **Devem validar novos valores para variáveis de instância.**
    - **Podem retornar um valor para indicar dados inválidos.**
- **Métodos *get*:**
  - **Também conhecidos como métodos de acesso ou métodos de consulta.**
  - **Obtêm os valores das variáveis de instância.**
  - **Podem controlar o formato dos dados que retornam.**

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.7

Se necessário, forneça métodos `public` para alterar e recuperar os valores de variáveis de instância `private`. Essa arquitetura ajuda a ocultar a implementação de uma classe dos seus clientes, o que aprimora a modificabilidade do programa.

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.8

Os projetistas de classe não precisam fornecer métodos *set* ou *get* para cada campo `private`. Essas capacidades devem ser fornecidas somente quando fizerem sentido.

© 2005 by Pearson Education do Brasil

8.7 Notas sobre os métodos *Set* e *Get* (*Continuação*)

- **Métodos predicados:**
  - **Testam se certa condição no objeto é verdadeira ou falsa e retornam o resultado.**
  - **Exemplo: Um método `isEmpty` para uma *classe contêiner* (uma classe capaz de conter muitos objetos).**
- **Encapsular tarefas específicas em métodos próprios simplifica os esforços de depuração.**

© 2005 by Pearson Education do Brasil

8.8 Composição

- **Composição:**
  - **Uma classe pode ter referências a objetos de outras classes como membros.**
  - **Às vezes conhecido como um relacionamento do tipo '*tem um*'.**

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.9

Uma forma de reutilização de software é a composição, em que uma classe tem como membros referências a objetos de outras classes.

© 2005 by Pearson Education do Brasil

Resumo

```java
// Fig. 8.7: Date.java
// Declaração da classe Date

public class Date
{
   private int month; // 1-12
   private int day;   // 1-31 conforme o mês
   private int year;  // qualquer ano

   // construtor: chama checkMonth para confirmar o valor adequado para month;
   // chama checkDay para confirmar o valor adequado para day
   public Date( int theMonth, int theDay, int theYear )
   {
      month = checkMonth( theMonth ); // valida month
      year = theYear; // poderia validar year
      day = checkDay( theDay ); // valida day

      System.out.printf(
         "Date object constructor for date %s\n", this );
   } // fim do construtor Date

```

Date.java

(1 de 3)

© 2005 by Pearson Education do Brasil

Resumo

Date.java

(2 de 3)

```
22    // método utilitário para confirmar o valor adequado de month
23    private int checkMonth( int testMonth )
24    {
25       if ( testMonth > 0 && testMonth <= 12 ) // valida month
26          return testMonth;
27       else // month é inválido
28       {
29          System.out.printf(
30             "Invalid month (%d) set to 1.", testMonth );
31          return 1; // mantém objeto em estado consistente
32       } // fim de else
33    } // fim do método checkMonth
34
35    // utilitário para confirmar o valor adequado de day com base em month e year
36    private int checkDay( int testDay )
37    {
38       int daysPerMonth[] =
39          { 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
40
```

Valida o valor do mês

Valida o valor do dia

© 2005 by Pearson Education do Brasil

Resumo

Date.java

(3 de 3)

```
41       // verifica se day está no intervalo para month
42       if ( testDay > 0 && testDay <= daysPerMonth[ month ] )
43          return testDay;
44
45       // verifica ano bissexto
46       if ( month == 2 && testDay == 29 && ( year % 400 == 0 ||
47            ( year % 4 == 0 && year % 100 != 0 ) ) )
48          return testDay;
49
50       System.out.printf( "Invalid day (%d) set to 1.", testDay );
51       return 1;  // mantém objeto em estado consistente
52    } // fim do método checkDay
53
54    // retorna uma String do formato mês/dia/ano
55    public String toString()
56    {
57       return String.format( "%d/%d/%d", month, day, year );
58    } // fim do método toString
59 } // fim da classe Date
```

Verifique se o dia é 29 de fevereiro em um ano bissexto

© 2005 by Pearson Education do Brasil

Resumo

Employee.java

```
// Fig. 8.8: Employee.java
// Classe Employee com referência a outros objetos.

public class Employee
{
   private String firstName;
   private String lastName;
   private Date birthDate;
   private Date hireDate;

   // construtor para inicializar nome, data de nascimento e data de contratação
   public Employee( String first, String last, Date dateOfBirth,
      Date dateOfHire )
   {
      firstName = first;
      lastName = last;
      birthDate = dateOfBirth;
      hireDate = dateOfHire;
   } // fim do construtor Employee

   // converte Employee em formato de String
   public String toString()
   {
      return String.format( "%s, %s  Hired: %s  Birthday: %s",
         lastName, firstName, hireDate, birthDate );
   } // fim do método toString
} // fim da classe Employee
```

Employee contém referências a dois objetos **Date**

Chamadas implícitas aos métodos **hireDate** e **toString** de **birthDate**

© 2005 by Pearson Education do Brasil

Resumo

EmployeeTest.java

```
// Fig. 8.9: EmployeeTest.java
// Demonstração de composição.

public class EmployeeTest
{
   public static void main( String args[] )
   {
      Date birth = new Date( 7, 24, 1949 );
      Date hire = new Date( 3, 12, 1988 );
      Employee employee = new Employee( "Bob", "Blue", birth, hire );

      System.out.println( employee );
   } // fim de main
} // fim da classe EmployeeTest
```

Cria um objeto **Employee**

Exibe o objeto **Employee**

```
Date object constructor for date 7/24/1949
Date object constructor for date 3/12/1988
Blue, Bob  Hired: 3/12/1988  Birthday: 7/24/1949
```

© 2005 by Pearson Education do Brasil

8.9 Enumerações

- **Tipos `enum`:**
  - **Declarados com uma declaração `enum`.**
    - **Uma lista separada por vírgulas de constantes `enum`.**
    - **Declara uma classe `enum` com as seguintes restrições:**
      - **tipos `enum` são implicitamente `final`;**
      - **constantes `enum` são implicitamente `static`; e**
      - **tentar criar um objeto de um tipo `enum` com `new` é um erro de compilação.**
  - **Constantes `enum` podem ser utilizadas em qualquer lugar em que as constantes podem ser utilizadas.**
  - **Construtor `enum`:**
    - **Como ocorre com construtores de classe, eles podem especificar parâmetros e podem ser sobrecarregados.**

© 2005 by Pearson Education do Brasil

Resumo

Book.java

(1 de 2)

```
// Fig. 8.10: Book.java
// Declarando um tipo enum com um construtor e campos de instância explícitos
// e métodos de de acesso para esses campos

public enum Book
{
   // declara constantes de tipo enum
   JHTP6( "Java How to Program 6e", "2005" ),
   CHTP4( "C How to Program 4e", "2004" ),
   IW3HTP3( "Internet & World Wide Web How to Program 3e", "2004" ),
   CPPHTP4( "C++ How to Program 4e", "2003" ),
   VBHTP2( "Visual Basic .NET How to Program 2e", "2002" ),
   CSHARPHTP( "C# How to Program", "2002" );

   // campos de instância
   private final String title; // título de livro
   private final String copyrightYear; // ano dos direitos autorais

   // construtor enum
   Book( String bookTitle, String year )
   {
      title = bookTitle;
      copyrightYear = year;
   } // fim do construtor enum Book

```

Declara seis constantes **enum**

Argumentos a serem passados para o construtor de **enum**

Declara variáveis de instância

Declara o construtor de **enum Book**

© 2005 by Pearson Education do Brasil

Resumo

```
   // método de acesso para título de campo
   public String getTitle()
   {
      return title;
   } // fim do método getTitle

   // método de acesso para o campo copyrightYear
   public String getCopyrightYear()
   {
      return copyrightYear;
   } // fim do método getCopyrightYear
} // fim do enum Book
```

Book.java

(2 de 2)

© 2005 by Pearson Education do Brasil

8.9 Enumerações (*Continuação*)

- **Método `static values`:**
  - **Gerado pelo compilador para cada `enum`.**
  - **Retorna um array das constantes de `enum` na ordem em que elas foram declaradas.**
- **Método `static range` da classe `EnumSet`:**
  - **Aceita dois parâmetros, a primeira e a última constantes `enum` no intervalo desejado.**
  - **Retorna um `EnumSet` que contém as constantes nesse intervalo, inclusive.**
  - **Uma instrução `for` aprimorada pode iterar por uma `EnumSet` assim como pode iterar por um array.**

© 2005 by Pearson Education do Brasil

Resumo

EnumTest.java

```java
// Fig. 8.11: EnumTest.java
// Testando tipo enum Book.
import java.util.EnumSet;

public class EnumTest
{
   public static void main( String args[] )
   {
      System.out.println( "All books:\n" );

      // imprime todos os livros em enum Book
      for ( Book book : Book.values() )
         System.out.printf( "%-10s%-45s%s\n", book,
             book.getTitle(), book.getCopyrightYear() );

      System.out.println( "\nDisplay a range of enum constants:\n" );

      // imprime os primeiros quatro livros
      for ( Book book : EnumSet.range( Book.JHTP6, Book.CPPHTP4 ) )
         System.out.printf( "%-10s%-45s%s\n", book,
             book.getTitle(), book.getCopyrightYear() );
   } // fim de main
} // fim da classe EnumTest
```

Loop **for** aprimorado itera por cada constante **enum** no array retornado pelo método **value**

Loop **for** aprimorado itera por cada constante **enum** no **EnumSet** retornado pelo método **range**

© 2005 by Pearson Education do Brasil

Resumo

```
All books:

JHTP6     Java How to Program 6e                        2005
CHTP4     C How to Program 4e                           2004
IW3HTP3   Internet & World Wide Web How to Program 3e   2004
CPPHTP4   C++ How to Program 4e                         2003
VBHTP2    Visual Basic .NET How to Program 2e           2002
CSHARPHTP C# How to Program                             2002

Display a range of enum constants:

JHTP6     Java How to Program 6e                        2005
CHTP4     C How to Program 4e                           2004
IW3HTP3   Internet & World Wide Web How to Program 3e   2004
CPPHTP4   C++ How to Program 4e                         2003
```

EnumTest.java

(2 de 2)

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.6

Em uma declaração `enum`, é um erro de sintaxe declarar constantes enum depois dos construtores, campos e métodos do tipo `enum` na declaração `enum`.

© 2005 by Pearson Education do Brasil

8.10 Coleta de lixo e o método `finalize`

- **Coleta de lixo:**
  - **A JVM marca um objeto para coleta de lixo quando não há nenhuma outra referência a esse objeto.**
  - **O coletor de lixo da JVM irá recuperar esses objetos da memória para que eles possam ser utilizados por outros objetos.**
- **Método `finalize`:**
  - **Todas as classes em Java têm o método `finalize`:**
    - **Herdado da classe `Object`.**
  - **`finalize` é chamado pelo coletor de lixo quando ele realiza a faxina de terminação.**
  - **`finalize` não recebe nenhum parâmetro e tem tipo de retorno `void`.**

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.10

Uma classe que utiliza recursos do sistema, como arquivos em disco, deve fornecer um método para finalmente liberar os recursos. Muitas classes da API do Java fornecem métodos `close` ou `dispose` para esse propósito.

Por exemplo, a classe `Scanner` (`java.sun.com/j2se/5.0/docs/api/java/util/Scanner.html`) tem um método `close`.

© 2005 by Pearson Education do Brasil

8.11 Membros da classe `static`

- Campos `static`:
  - Também conhecidos como variáveis de classe.
  - Representam informações de ‘escopo de classe’.
  - Utilizados quando:
    - todos os objetos da classe devem compartilhar a mesma cópia dessa variável de instância; ou
    - essa variável de instância deve ser acessível mesmo quando não existir nenhum objeto da classe.
  - Podem ser acessados com o nome da classe ou com o nome de um objeto e um ponto (`.`).
  - Devem ser inicializados nas suas declarações ou, caso contrário, o compilador irá inicializá-los com um valor-padrão (`0` para `int`s).

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.11

Utilize uma variável `static` quando todos os objetos de uma classe precisarem utilizar a mesma cópia da variável.

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.12

Variáveis e métodos de classe `static` existem e podem ser utilizados, mesmo se nenhum objeto dessa classe tiver sido instanciado.

© 2005 by Pearson Education do Brasil

Resumo

Employee.java

(1 de 2)

```
// Fig. 8.12: Employee.java
// Variável estática utilizada para manter uma contagem do número de
// Objetos Employee na memória.

public class Employee
{
   private String firstName;
   private String lastName;
   private static int count = 0; // número de objetos na memória

   // inicializa Employee, adiciona 1 a static count e
   // gera a saída de String indicando que o construtor foi chamado
   public Employee( String first, String last )
   {
      firstName = first;
      lastName = last;

      count++;  // incrementa contagem estática de empregados
      System.out.printf( "Employee constructor: %s %s; count = %d\n",
         firstName, lastName, count );
   } // fim do construtor Employee

```

Declara um campo **static**

Incrementa um campo **static**

© 2005 by Pearson Education do Brasil

Resumo

Employee.java

(2 de 2)

```
   // subtrai 1 de static count quando o coletor de lixo
   // chama finalize para limpar o objeto;
   // confirma se finalize foi chamado
   protected void finalize()
   {
      count--; // decrementa contagem estática de empregados
      System.out.printf( "Employee finalizer: %s %s; count = %d\n",
         firstName, lastName, count );
   } // fim do método finalize

   // obtém nome
   public String getFirstName()
   {
      return firstName;
   } // fim do método getFirstName

   // obtém sobrenome
   public String getLastName()
   {
      return lastName;
   } // fim do método getLastName

   // método static para obter  valor de contagem de estática
   public static int getCount()
   {
      return count;
   } // fim do método getCount
} // fim da classe Employee
```

Declara o método **`finalize`**

Declara o método **`getCount`** como **`static`** para obter o campo **`static field`**

© 2005 by Pearson Education do Brasil

Resumo

```
// Fig. 8.13: EmployeeTest.java
// Demonstração do membro static.

public class EmployeeTest
{
   public static void main( String args[] )
   {
      // mostra que a contagem é 0 antes de criar Employees
      System.out.printf( "Employees before instantiation: %d\n",
         Employee.getCount() );

      // cria dois Employees; a co
      Employee e1 = new Employee( "Susan", "Baker" );
      Employee e2 = new Employee( "Bob", "Blue" );

```

Chama o método **`static getCount`** utilizando o nome da classe **`Employee`**

Cria novos objetos **`Employee`**

`EmployeeTest.java`

(1 de 3)

© 2005 by Pearson Education do Brasil

Resumo

EmployeeTest.j

(2 de 3)

```
16      // mostra que a contagem é 2 depois de criar dois Employees
17      System.out.println( "\nEmployees after instantiation: " );
18      System.out.printf( "via e1.getCount(): %d\n", e1.getCount() );
19      System.out.printf( "via e2.getCount(): %d\n", e2.getCount() );
20      System.out.printf( "via Employee.getCount(): %d\n",
21         Employee.getCount() );
22
23      // obtém nomes de Employees
24      System.out.printf( "\nEmployee 1: %s %s\nEmployee 2: %s %s\n\n",
25         e1.getFirstName(), e1.getLastName(),
26         e2.getFirstName(), e2.getLastName() );
27
28      // nesse exemplo, há somente uma referência a cada Employee,
29      // ssim as duas instruções a seguir fazem com que a JVM marque cada
30      // objeto Employee para coleta de lixo
31      e1 = null;
32      e2 = null;
33
34      System.gc(); // pede que a coleta de lixo ocorra agora
35
```

Chama o método **static** **getCount** fora dos objetos

Chama o método **static** **getCount** dentro dos objetos

Remove referências aos objetos, a JVM irá marcá-las para coleta de lixo

Chama o método **static** **gc** da classe **System** para indicar que a coleta de lixo deve ser tentada

© 2005 by Pearson Education do Brasil

Resumo

EmployeeTest.java

(3 de 3)

```
      // mostra a contagem de Employee depois de chamar o coletor de lixo; contagem
      // ida pode ser 0, 1 ou 2 com base na execução do coletor de lixo
      // imediata e número de objetos Employees coletados
      System.out.printf( "\nEmployees after System.gc(): %d\n",
         Employee.getCount() );
   } // fim de main
} // fim da EmployeeTest
```

Chama o método **static getCount**

```
Employees before instantiation: 0
Employee constructor: Susan Baker; count = 1
Employee constructor: Bob Blue; count = 2

Employees after instantiation:
via e1.getCount(): 2
via e2.getCount(): 2
via Employee.getCount(): 2

Employee 1: Susan Baker
Employee 2: Bob Blue

Employee finalizer: Bob Blue; count = 1
Employee finalizer: Susan Baker; count = 0

Employees after System.gc(): 0
```

© 2005 by Pearson Education do Brasil

Boa prática de programação 8.1

Invoque cada método `static` utilizando o nome de classe e um ponto (`.`) para enfatizar que o método sendo chamado é um método `static`.

© 2005 by Pearson Education do Brasil

8.11 Membros da classe `static` (*Continuação*)

- Objetos `String` são imutáveis:
  - Operações de concatenação de string na verdade resultam na criação de um novo objeto `String`.
- Método `static` `gc` da classe `System`:
  - Indica que o coletor de lixo deve tentar da melhor maneira possível reivindicar objetos elegíveis para coleta de lixo.
  - É possível que nenhum objeto ou somente um subconjunto de objetos elegíveis seja coletado.
- Métodos `static` não podem acessar membros de classe não-`static`:
  - Também não podem utilizar a referência `this`.

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.7

Um erro de compilação ocorre se um método `static` chamar um método de uma instância (não-`static`) na mesma classe utilizando somente o nome do método.

De maneira semelhante, um erro de compilação ocorre se um método `static` tentar acessar uma variável de instância na mesma classe utilizando somente o nome de variável.

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.8

Referenciar `this` em um método `static` é um erro de sintaxe.

© 2005 by Pearson Education do Brasil

8.12 Import `static`

- **Declarações de importação `static`:**
  - **Permitem que os programadores façam referência a membros `static` importados como se eles fossem declarados na classe que os utiliza.**
  - **import `static` simples:**
    - **import `static` `nomeDoPacote.NomeDaClasse.nomeDoMembroStatic`;**
  - **import `static` por demanda:**
    - **import `static` `nomeDoPacote.NomeDaClasse.*`;**
    - **Importa todos os membros `static` da classe especificada.**

© 2005 by Pearson Education do Brasil

Resumo

StaticImportTest

.java

```
// Fig. 8.14: StaticImportTest.java
// Utilizando import static para importar métodos static da classe Math.
import static java.lang.Math.*;

public class StaticImportTest
{
   public static void main( String args[] )
   {
      System.out.printf( "sqrt( 900.0 ) = %.1f\n", sqrt( 900.0 ) );
      System.out.printf( "ceil( -9.8 ) = %.1f\n", ceil( -9.8 ) );
      System.out.printf( "log( E ) = %.1f\n", log( E ) );
      System.out.printf( "cos( 0.0 ) = %.1f\n", cos( 0.0 ) );
   } // fim de main
} // fim da classe StaticImportTest
```

import **static** por demanda

Utiliza os métodos **static** e variáveis de instância de **Math** sem precedê-los com **Math**.

```
sqrt( 900.0 ) = 30.0
ceil( -9.8 ) = -9.0
log( E ) = 1.0
cos( 0.0 ) = 1.0
```

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.9

Um erro de compilação ocorre se um programa tentar importar métodos `static` que têm a mesma assinatura ou campos `static` que têm o mesmo nome proveniente de duas ou mais classes.

© 2005 by Pearson Education do Brasil

8.13 Variáveis de instância `final`

- **Princípio do menor privilégio:**
  - **O código só deve ter o privilégio e acesso necessário para realizar sua tarefa e nada mais.**
- **Variáveis de instância `final`:**
  - **Palavra-chave `final`:**
    - **Especifica que uma variável não é modificável (é uma constante).**
  - **Variáveis de instância `final` podem ser inicializadas nas suas declarações.**
    - **Se não forem inicializadas nas suas declarações, elas deverão ser inicializadas em todos os construtores.**

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.13

Declarar uma variável de instância como `final` ajuda a impor o princípio do menor privilégio.

Se uma variável de instância não deve ser modificada, declare-a como sendo `final` para evitar modificação.

© 2005 by Pearson Education do Brasil

Resumo

Increment.java

```java
// Fig. 8.15: Increment.java
// Variável de instância final em uma classe.

public class Increment
{
   private int total = 0; // total de todos os incrementos
   private final int INCREMENT; // variável constante (não-inicializada)

   // construtor inicializa variável de instância final INCREMENT
   public Increment( int incrementValue )
   {
      INCREMENT = incrementValue; // inicializa variável constante (uma vez)
   } // fim do construtor Increment

   // adiciona INCREMENT ao total
   public void addIncrementToTotal()
   {
      total += INCREMENT;
   } // fim do método addIncrementToTotal

   // retorna representação de String dos dados de um objeto Increment
   public String toString()
   {
      return String.format( "total = %d", total );
   } // fim do método toIncrementString
} // fim da classe Increment
```

Declara a variável de instância **final**

Inicialize a variável de instância **final** dentro de um construtor

© 2005 by Pearson Education do Brasil

Resumo

```
// Fig. 8.16: IncrementTest.java
// Variável final inicializada com um argumento de construtor.

public class IncrementTest
{
   public static void main( String args[] )
   {
      Increment value = new Increment( 5 );

      System.out.printf( "Before incrementing: %s\n\n", value );

      for ( int i = 1; i <= 3; i++ )
      {
         value.addIncrementToTotal();
         System.out.printf( "After increment %d: %s\n", i, value );
      } // fim de for
   } // fim de main
} // fim da classe IncrementTest
```

IncrementTest.

Cria um objeto **Increment**

Chama o método **addIncrementToTotal**

```
Before incrementing: total = 0

After increment 1: total = 5
After increment 2: total = 10
After increment 3: total = 15
```

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.10

Tentar modificar uma variável de instância `final` depois que é ela inicializada é um erro de sintaxe.

© 2005 by Pearson Education do Brasil

Dica de prevenção de erro 8.2

Tentativas de modificar uma variável de instância `final` são capturadas em tempo de compilação em vez de causar erros em tempo de execução.

Sempre é preferível remover bugs em tempo de compilação, se possível, em vez de permitir que eles passem para o tempo de execução (em que estudos descobriram que o custo do reparo é freqüentemente muito mais caro).

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.14

Um campo `final` também deve ser declarado `static` se for inicializado na sua declaração. Depois que um campo `final` é inicializado na sua declaração, seu valor nunca pode mudar. Portanto, não é necessário criar uma cópia separada do campo para cada objeto da classe. Criar o campo `static` permite que todos os objetos da classe compartilhem o campo `final`.

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.11

Não inicializar uma variável de instância `final` na sua declaração ou em cada construtor da classe produz um erro de compilação indicando que a variável talvez não tenha sido inicializada. O mesmo erro ocorre se a classe inicializar a variável em alguns, mas não em todos, construtores da classe.

© 2005 by Pearson Education do Brasil

Resumo

```
Increment.java:13: variable INCREMENT might not have been initialized
   } // end Increment constructor
   ^
1 error
```

Increment.java

© 2005 by Pearson Education do Brasil

8.14 Capacidade de reutilização de software

- **Desenvolvimento rápido de aplicações:**
  - **A capacidade de reutilização de software acelera o desenvolvimento de softwares de alta qualidade e poderosos.**
- **A API do Java:**
  - **É uma estrutura em que os desenvolvedores Java podem trabalhar para conseguir verdadeira reutilização e rápido desenvolvimento de aplicações.**
  - **Documentação:**
    - **java.sun.com/j2se/5.0/docs/api/index.html**
      - **Ou java.sun.com/j2se/5.0/download.html para download.**

© 2005 by Pearson Education do Brasil

8.15 Abstração de dados e encapsulamento

- **Abstração de dados:**
  - **Ocultamento de informações.**
    - **Normalmente, classes ocultam os detalhes de implementação dos seus clientes.**
  - **Tipos de dados abstratos (Abstract Data Type – ADTs):**
    - **Representação de dados:**
      - **Exemplo: Tipo primitivo `int` é uma representação abstrata de um inteiro.**
        - **`ints` são apenas aproximações de inteiros; podem produzir estouros aritméticos.**
    - **Operações que podem ser realizadas nos dados.**

© 2005 by Pearson Education do Brasil

Boa prática de programação 8.2

Evite reinventar a roda. Estude as capacidades da API do Java. Se a API contiver uma classe que atenda aos requisitos do seu programa, utilize essa classe em vez de criar uma própria.

© 2005 by Pearson Education do Brasil

8.15 Abstração de dados e encapsulamento (*Continuação*)

- **Filas:**
  - **Semelhante a uma fila convencional:**
    - **Os clientes posicionam itens na fila (enfileiram um item).**
    - **Os clientes obtêm itens de volta a partir da fila (desenfileiram um item).**
    - **Ordem primeiro a entrar, primeiro a sair (first-in, first-out — FIFO).**
  - **A representação interna dos dados permanece oculta:**
    - **Os clientes só vêem a capacidade de enfileirar e desenfileirar itens.**

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.15

Programadores criam tipos por meio do mecanismo de classe. Novos tipos podem ser projetados a fim de serem convenientes de utilizar como tipos predefinidos. Isso marca o Java como uma linguagem extensível. Embora a linguagem seja fácil de estender via os novos tipos, o programador não pode alterar a base da linguagem por conta própria.

© 2005 by Pearson Education do Brasil

8.16 Estudo de caso da classe `Time`: Controlando o acesso a membros

- **Para declarar uma classe como reutilizável:**
  - **Declare uma classe `public`.**
  - **Adicione uma declaração `package` ao arquivo de código-fonte:**
    - **deve ser a primeira instrução executável no arquivo;**
    - **o nome do `package` deve consistir no seu nome de domínio na Internet na ordem inversa seguido por outros nomes para o pacote.**
      - **Exemplo: `com.deitel.jhtp6.ch08`**
      - **O nome do `package` é parte do nome completamente qualificado de classe.**
        - **Distingue entre múltiplas classes que têm o mesmo nome, mas pertencem a pacotes diferentes.**
        - **Evita o conflito de nomes (também chamado *colisão de nomes*).**
      - **O nome da classe sem um nome do `package` é o nome simples.**

© 2005 by Pearson Education do Brasil

Resumo

Time1.java

```
// Fig. 8.18: Time1.java
// Declaração de classe Time1 mantém a data/hora no formato de 24 horas.
package com.deitel.jhtp6.ch08;

public class Time1
{
   private int hour;   // 0 - 23
   private int minute; // 0 - 59
   private int second; // 0 - 59

   // configura um novo valor de data/hora usando data/hora universal; realiza
   // testes de validade nos dados; configura valores inválidos como zero
   public void setTime( int h, int m, int s )
   {
      hour = ( ( h >= 0 && h < 24 ) ? h : 0 );   // valida horas
      minute = ( ( m >= 0 && m < 60 ) ? m : 0 ); // valida minutos
      second = ( ( s >= 0 && s < 60 ) ? s : 0 ); // valida segundos
   } // fim do método setTime

```

Declaração **package**

Time1 é uma classe **public**, portanto ela pode ser utilizada pelos importadores desse pacote

(1 de 2)

© 2005 by Pearson Education do Brasil

Resumo

Time1.java

(2 de 2)

```
   // converte em String no formato de data/hora universal (HH:MM:SS)
   public String toUniversalString()
   {
      return String.format( "%02d:%02d:%02d", hour, minute, second );
   } // fim do método toUniversalString

   // converte em String no formato padrão de data/hora (H:MM:SS AM ou PM)
   public String toString()
   {
      return String.format( "%d:%02d:%02d %s",
         ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 ),
         minute, second, ( hour < 12 ? "AM" : "PM" ) );
   } // fim do método toString
} // fim da classe Time1
```

© 2005 by Pearson Education do Brasil

8.16 Estudo de caso da classe `Time`: Controlando o acesso a membros (*Cont.*)

- **Compile a classe de modo que ela seja colocada na estrutura apropriada de diretórios de pacotes.**
  - **Exemplo: nosso pacote deveria estar no diretório:**

```
com
  └──► deitel
         └──► jhtp6
                └──► ch08
```

  - **Opção `-d` da linha de comando `javac`:**
    - **`javac` cria diretórios apropriados com base na declaração `package` da classe.**
    - **Um ponto (`.`) depois de `-d` representa o diretório atual.**

© 2005 by Pearson Education do Brasil

8.16 Estudo de caso da classe `Time`: Controlando o acesso a membros (*Cont.*)

- Importe a classe reutilizável em um programa:
  - Declaração `import` do tipo simples.
    - Importa uma classe única.
    - Exemplo: `import java.util.Random;`
  - Declaração `import` do tipo por demanda:
    - Importa todas as classes em um pacote.
    - Exemplo: `import java.util.*;`

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.12

Utilizar a declaração `import java.*;` resulta em um erro de compilação. Você deve especificar o nome exato do pacote do qual você quer importar classes.

© 2005 by Pearson Education do Brasil

Resumo

```
// Fig. 8.19: Time1PackageTest.java
// Objeto Time1 utilizado em um aplicativo.
import com.deitel.jhtp6.ch08.Time1; // importa classe Time1

public class Time1PackageTest
{
   public static void main( String args[] )
   {
      // cria e inicializa um objeto Time1
      Time1 time = new Time1(); // chama o construtor Time1

      // output string representations of the time
      System.out.print( "The initial universal time is: " );
      System.out.println( time.toUniversalString() );
      System.out.print( "The initial standard time is: " );
      System.out.println( time.toString() );
      System.out.println(); // gera saída de uma linha em branco

```

Declaração **`import`** do tipo simples

Referência à classe **`Time1`** pelo seu nome simples

`Time1PackageTest`

`.java`

© 2005 by Pearson Education do Brasil

Resumo

Time1PackageTest.java

(2 de 2)

```
19      // altera a data/hora e gera saída da data/hora atualizada
20      time.setTime( 13, 27, 6 );
21      System.out.print( "Universal time after setTime is: " );
22      System.out.println( time.toUniversalString() );
23      System.out.print( "Standard time after setTime is: " );
24      System.out.println( time.toString() );
25      System.out.println(); // gera saída de uma linha em branco
26
27      // configura data/hora com valores inválidos; gera saída da data/hora atualizada
28      time.setTime( 99, 99, 99 );
29      System.out.println( "After attempting invalid settings:" );
30      System.out.print( "Universal time: " );
31      System.out.println( time.toUniversalString() );
32      System.out.print( "Standard time: " );
33      System.out.println( time.toString() );
34    } // fim de main
35 } // fim da classe Time1PackageTest
```

```
The initial universal time is: 00:00:00
The initial standard time is: 12:00:00 AM

Universal time after setTime is: 13:27:06
Standard time after setTime is: 1:27:06 PM

After attempting invalid settings:
Universal time: 00:00:00
Standard time: 12:00:00 AM
```

© 2005 by Pearson Education do Brasil

8.16 Estudo de caso da classe `Time`: Controlando o acesso a membros (*Cont.*)

- **Carregador de classe:**
  - **Localiza classes de que o compilador precisa:**
    - **Primeiro pesquisa as classes Java padrão empacotadas no JDK.**
    - **Depois procura pacotes opcionais.**
      - **Essas são permitidas pelo mecanismo de extensão do Java.**
    - **Por fim, pesquisa o `classpath`:**
      - **Lista de diretórios ou repositórios de arquivos separados por separadores de diretório.**
        - **Esses arquivos normalmente terminam com `.jar` ou `.zip`.**
        - **Classes-padrão estão no repositório de arquivos `rt.jar`.**

© 2005 by Pearson Education do Brasil

8.16 Estudo de caso da classe `Time`: Controlando o acesso a membros (*Cont.*)

- **Por padrão, o classpath consiste apenas no diretório atual. Mas ele pode ser modificado:**
  - **fornecendo a opção `-classpath` para o compilador `javac`; ou**
  - **configurando a variável de ambiente `CLASSPATH`.**
- **A JVM deve localizar as classes da mesma maneira como o compilador localiza:**
  - **O comando `java` pode utilizar outros `classpaths` com as mesmas técnicas utilizadas pelo comando `javac`.**

© 2005 by Pearson Education do Brasil

Erro comum de programação 8.13

Especificar um classpath explícito elimina o diretório atual do classpath. Isso impede que classes no diretório atual (incluindo pacotes no diretório atual) sejam carregadas adequadamente. Se classes precisarem ser carregadas do diretório atual, inclua um ponto (.) ao classpath para especificar o diretório atual.

© 2005 by Pearson Education do Brasil

Observação de engenharia de software 8.16

Em geral, uma prática melhor é utilizar a opção `-classpath` do compilador, em vez da variável de ambiente `CLASSPATH`, para especificar o classpath para um programa. Isso permite que cada aplicação tenha seu próprio classpath.

© 2005 by Pearson Education do Brasil

Dica de prevenção de erro 8.3

Especificar o classpath com a variável de ambiente `CLASSPATH` pode resultar em erros sutis e difíceis de localizar em programas que utilizam diferentes versões do mesmo pacote.

© 2005 by Pearson Education do Brasil

8.17 Acesso de pacote

- **Acesso de pacote:**
  - **Métodos e variáveis declaradas sem nenhum modificador recebem acesso de pacote.**
  - **Isso não tem nenhum efeito se o programa consistir em uma única classe.**
  - **Isso tem efeito se o programa contiver múltiplas classes no mesmo pacote.**
    - **Membros com acesso de pacote podem ser acessados diretamente por meio de referências apropriadas aos objetos nas outras classes que pertencem ao mesmo pacote.**

© 2005 by Pearson Education do Brasil

Resumo

```
// Fig. 8.20: PackageDataTest.java
// Membros de acesso de pacote de uma classe permanecem acessíveis a outras
// classes no mesmo pacote.

public class PackageDataTest
{
   public static void main( String args[] )
   {
      PackageData packageData = new PackageData();

      // gera saída da representação String de packageData
      System.out.printf( "After instantiation:\n%s\n", packageData );

      // muda os dados de acesso de pacote no objeto packageData
      packageData.number = 77;
      packageData.string = "Goodbye";

      // gera saída da representação String de packageData
      System.out.printf( "\nAfter changing values:\n%s\n", packageData );
   } // fim de main
} // fim da classe PackageDataTest
```

Pode acessar diretamente membros com acesso de pacote

PackageDataTest

.java

© 2005 by Pearson Education do Brasil

Resumo

```
// classe com variáveis de instância de acesso de pacote
class PackageData
{
   int number; // variável de instância de acesso de pacote
   String string; // variável de instância de acesso de pacote

   // construtor
   public PackageData()
   {
      number = 0;
      string = "Hello";
   } // fim do construtor PackageData

   // retorna a representação String do objeto PackageData
   public String toString()
   {
      return String.format( "number: %d; string: %s", number, string );
   } // fim do método toString
} // fim da classe PackageData
```

Variável de instância de acesso de pacote

```
After instantiation:
number: 0; string: Hello

After changing values:
number: 77; string: Goodbye
```

PackageDataTest.java

(2 de 2)

© 2005 by Pearson Education do Brasil

8.19 (Opcional) Estudo de caso de engenharia de software: Começando a programar as classes do sistema ATM

- **Visibilidade:**
  - **Normalmente, atributos devem ser privados, e os métodos invocados pelos clientes devem ser públicos.**
  - ***Marcadores de visibilidade* em UML:**
    - **um sinal de adição (+) indica visibilidade pública; e**
    - **um sinal de subtração (-) indica visibilidade privada.**
- **Navegabilidade:**
  - **As *setas de navegabilidade* indicam em qual direção uma associação pode ser atravessada.**
  - **Navegabilidade *bidirecional*:**
    - **Associações com setas de navegabilidade nas duas extremidades ou nenhuma seta de navegabilidade pode ser atravessada em qualquer direção.**

© 2005 by Pearson Education do Brasil

8.19 Começando a programar as classes do sistema ATM (*Cont.*)

- **Implementando o sistema ATM a partir do seu design em UML (para cada classe):**
  - **Declare uma classe `public` com o nome no primeiro compartimento e um construtor sem argumentos vazio.**
  - **Declare variáveis de instância com base nos atributos no segundo compartimento.**
  - **Declare referências a outros objetos com base nas associações descritas no diagrama de classes.**
  - **Declare os shells dos métodos com base nas operações no terceiro compartimento.**
    - **Utilize o tipo de retorno `void` se nenhum tipo de retorno tiver sido especificado.**

© 2005 by Pearson Education do Brasil

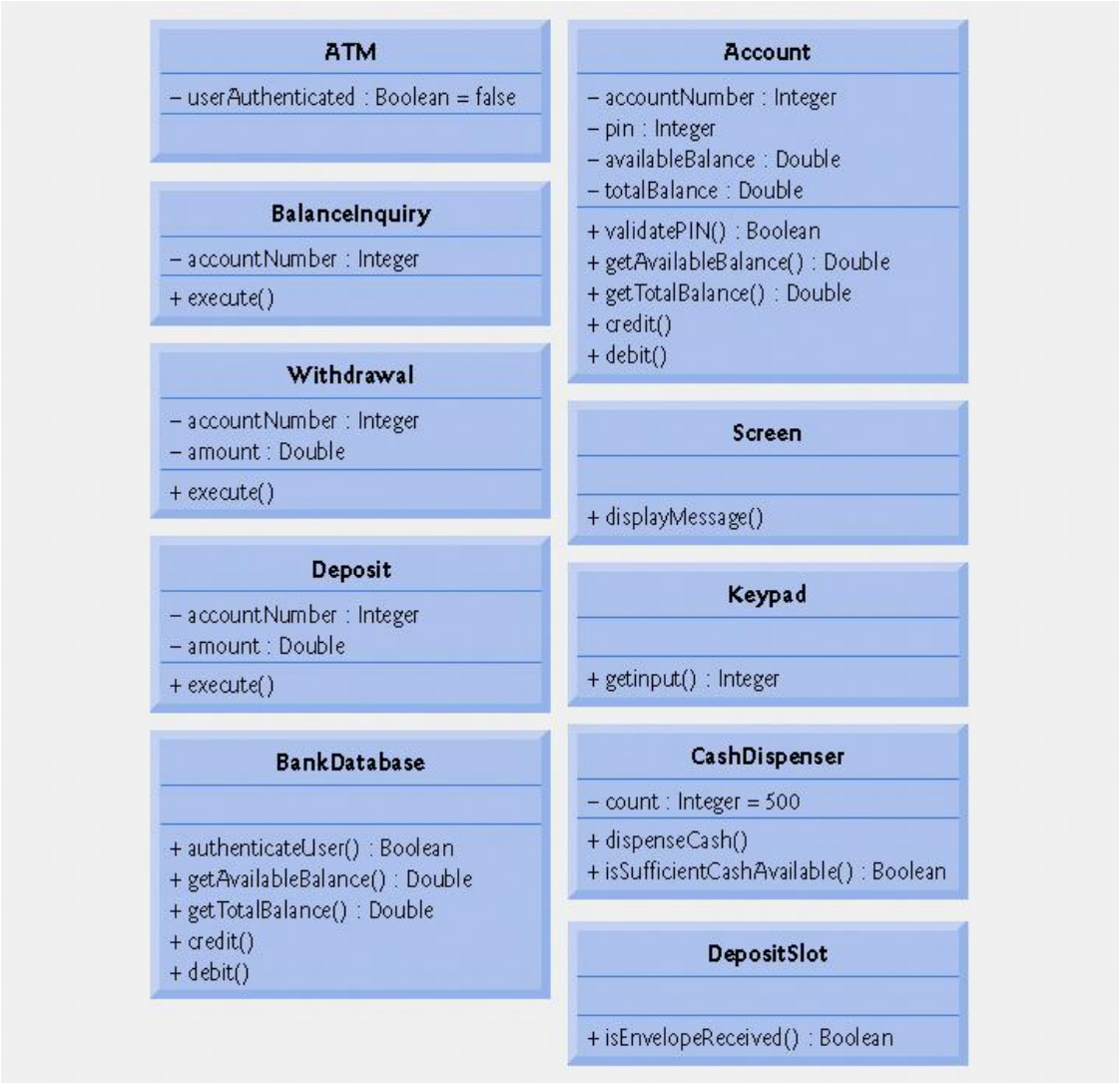

Figura 8.24 | Diagrama de classes com marcadores de visibilidade.

© 2005 by Pearson Education do Brasil

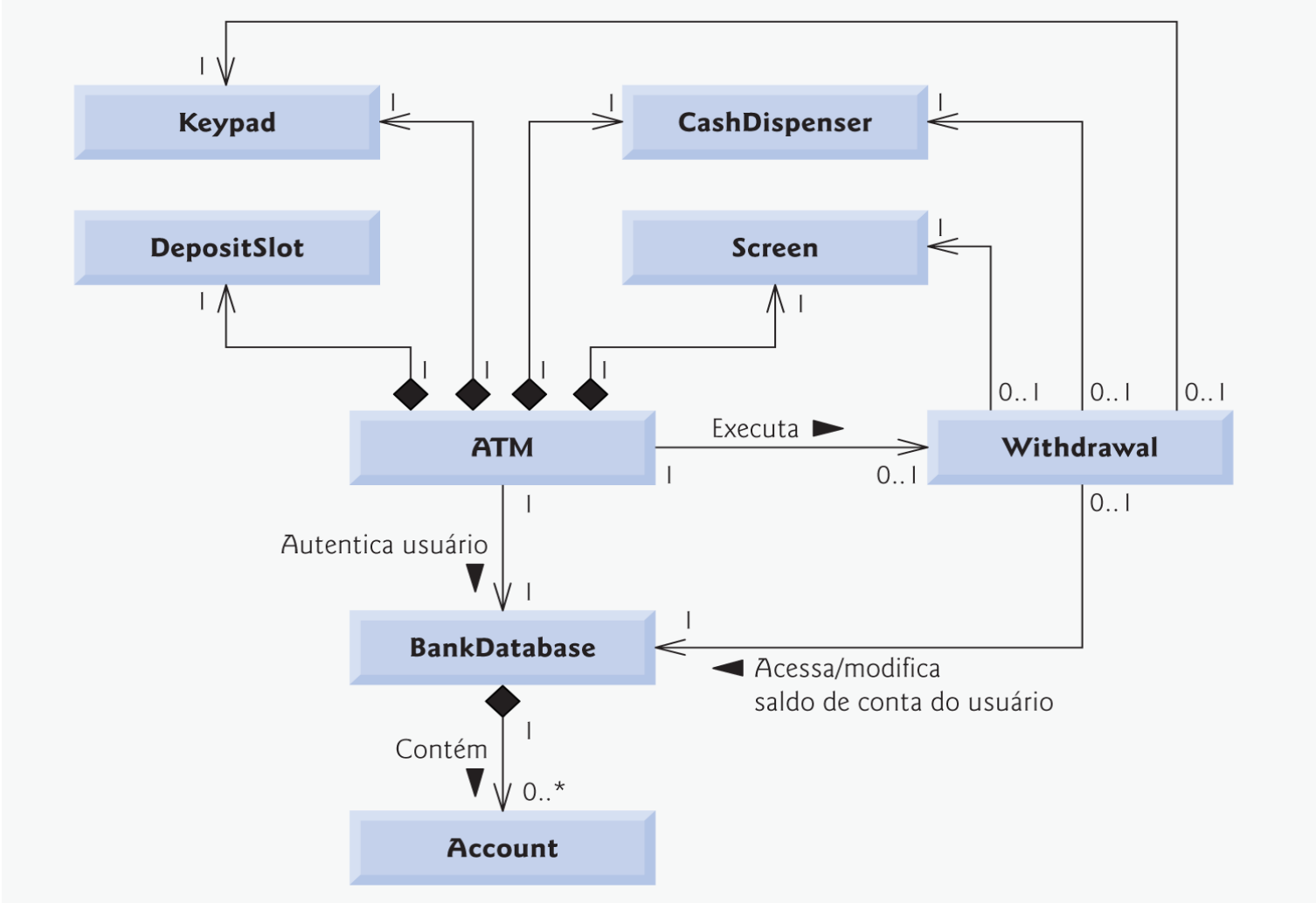

Figura 8.25 | Diagrama de classe com setas de navegabilidade.

© 2005 by Pearson Education do Brasil

Outline

withdrawal.java

```
// Classe Withdrawal representa uma transação de saque no ATM
public class Withdrawal
{
   // construtor sem argumento
   public Withdrawal()
   {
   } // fim do construtor sem argumentos Withdrawal
} // fim da classe Withdrawal
```

Classe para **Withdrawal**

Construtor sem argumento vazio

© 2005 by Pearson Education do Brasil

Resumo

```
1  // A classe Withdrawal representa uma transação de saque no ATM
2  public class Withdrawal
3  {
4     // atributos
5     private int accountNumber; // conta da qual a sacar fundos
6     private double amount; // quantia a retirar
7
8     // construtor sem argumento
9     public Withdrawal()
10    {
11    } // fim do construtor sem argumentos Withdrawal
12 } // fim da classe Withdrawal
```

Declara variáveis de instância

withdrawal.java

© 2005 by Pearson Education do Brasil

Resumo

withdrawal.java

```
1  // A classe Withdrawal representa uma transação de saque no ATM
2  public class Withdrawal
3  {
4     // atributos
5     private int accountNumber; // conta a sacar fundos
6     private double amount; // quantia a sacar
7
8     // referências a objetos associados
9     private Screen screen; // Tela do ATM
10    private Keypad keypad; // Teclado do ATM
11    private CashDispenser cashDispenser; // dispensador de cédulas do ATM
12    private BankDatabase bankDatabase; // banco de dados de informações sobre a conta
13
14    // construtor sem argumentos
15    public Withdrawal()
16    {
17    } // fim do construtor Withdrawal sem argumentos
18 } // fim da classe Withdrawal
```

Declara referências a outros objetos

© 2005 by Pearson Education do Brasil

Resumo

withdrawal.java

```
1  // A classe Withdrawal representa uma transação de saque no ATM
2  public class Withdrawal
3  {
4     // atributos
5     private int accountNumber; // conta da qual sacar fundos
6     private double amount; // quantia a sacar
7
8     // referências a objetos associados
9     private Screen screen; // Tela do ATM
10    private Keypad keypad; // Teclado do ATM
11    private CashDispenser cashDispenser; // Dispensador de cédulas do ATM
12    private BankDatabase bankDatabase; // banco de dados de informações sobre a conta
13
14    // construtor sem argumentos
15    public Withdrawal()
16    {
17    } // fim do construtor sem argumentos Withdrawal
18
19    // operações
20    public void execute()
21    {
22    } // fim do método execute
23 } // fim da classe Withdrawal
```

Declara um shell de um método com um tipo de retorno **void**

© 2005 by Pearson Education do Brasil

Resumo

```
// A classe Keypad representa o teclado de um ATM
public class Keypad
{
   // nenhum atributo foi especificado ainda

   // construtor sem argumentos
   public Keypad()
   {
   } // fim do construtor Keypad sem argumentos

   // operações
   public int getInput()
   {
   } // fim do método getInput
} // fim da classe Keypad
```

withdrawal.java

© 2005 by Pearson Education do Brasil